

Carlos José Requena Jiménez

Solutions Architect | Enterprise Architecture & Cloud Strategy

Palma, Spain · cjrequena@outlook.es · +34 669 258 306

linkedin.com/in/cjrequena · github.com/cjrequena

SUMMARY

Solutions Architect with 15+ years of experience leading cloud transformation, distributed systems architecture, and large-scale modernization initiatives in enterprise environments. Proven track record defining architecture strategy, scaling AWS-native platforms, and aligning engineering execution with business objectives.

Hands-on across the full architectural stack: event-driven and event-sourced distributed systems, CQRS and hexagonal service design, multi-tenant SaaS platforms with database-enforced tenant isolation, and AWS environments defined entirely as code across a multi-account organization. Equally comfortable setting the standard and writing the reference implementation — API contracts, messaging semantics, deployment topology, identity and authorization models, and the CI/CD pipelines that carry them to production.

Experienced driving cross-functional technical leadership, architecture governance, and high-availability systems supporting global operations and business-critical platforms, with a consistent emphasis on decision records, reusable patterns, cost-aware design, and mentoring the engineers who inherit the architecture.

CORE COMPETENCIES

Enterprise & Solution Architecture • Cloud Transformation Strategy • Architecture Governance & Decision Records (ADRs) • Technical Leadership • Architecture & Design Reviews • Reference Architectures & Standards • Trade-off and Build-vs-Buy Analysis • Scalability & Risk Assessment • Cost Optimization / FinOps • Stakeholder Management • Cross-Functional Influence (Product, Security, DevOps) • Engineering Mentorship • Agile / Scrum • Enterprise Architecture Management (LeanIX)

TECHNICAL EXPERTISE

Architecture & Design Patterns — Microservices • Hexagonal Architecture (Ports & Adapters) • CQRS • Event Sourcing • Domain-Driven Design (aggregates, value objects, domain events) • Event-Driven Architecture • API-First & Versioned REST Design • Multi-Tenancy & Tenant Isolation • Saga / Compensating Transactions • Idempotency & Optimistic Concurrency • Projections & Read Models • Server-Sent Events • Backend-for-Frontend • High Availability, Resilience & Graceful Degradation

Cloud & AWS — ECS Fargate • API Gateway (HTTP API v2) • VPC Link • Application Load Balancer • Cloud Map Service Discovery • Application Auto Scaling • Cognito (OIDC/JWT, custom claims, M2M client credentials) • Lambda • RDS PostgreSQL & RDS Proxy • ElastiCache Redis • MSK / Managed Kafka • S3 • Secrets Manager • ECR • Amplify (SSR hosting) • ACM • CloudWatch • VPC & Network Segmentation • Organizations, SCPs & IAM Identity Center • Multi-Account Landing Zones

Infrastructure as Code & Platform Engineering — Terraform (reusable modules, thin per-environment root stacks, remote-state composition, S3/DynamoDB backends) • Multi-Account Strategy & Account Vending • AWS SAM • Docker & Docker Compose • Ansible • Resource Naming & Tagging Standards • Environment Promotion & Release Engineering

CI/CD & DevOps — GitLab CI (parent/child pipelines, path-based change gating, DAG needs orchestration) • OIDC Keyless Cloud Authentication (no static credentials) • Plan-then-Apply Governance • Manual Promotion Gates (test → staging → production) • SemVer & CalVer Release Strategy • Artifact Registries (Maven, ECR) • GitHub Flow / Trunk-Based Development • Rolling Container Deployments • Pipeline Cost Control

Backend Engineering — Java 21 • Spring Boot 3.5 • Spring Cloud Stream • Spring Security (OAuth2 Resource Server, method security) • Spring Data JPA / Hibernate • QueryDSL • MapStruct • Flyway • RSQL Filtering • ShedLock (distributed scheduling) • OpenAPI / springdoc • Micrometer & Distributed Tracing • Maven (dependency management, shared parent POMs, published libraries) • Python

Data & Messaging — PostgreSQL (Row-Level Security, GiST exclusion constraints, PostGIS, transaction-scoped tenant context) • Event Store Design (append-only log, snapshots, durable subscription offsets) • Apache Kafka / MSK • CloudEvents • Dead-Letter Queues & Replay Strategies • Redis & Caffeine Caching • Schema Migration Governance • Data Modeling for Multi-Tenant Systems

Frontend & Full-Stack — TypeScript • Next.js (App Router, SSR / React Server Components) • React • TanStack Query • Zustand • Tailwind CSS • shadcn/ui • Internationalization & Multi-Market Routing (next-intl) • Server-Sent Events Consumption • Timezone-Correct UX • Accessibility

Security & Compliance — OAuth 2.0 / OIDC • JWT & Claim-Based Authorization • RBAC & Fine-Grained Permission Models • Database-Enforced Tenant Isolation • Least-Privilege IAM • Service Control Policy Guardrails • Secrets Management • Service-to-Service Trust (machine-to-machine) • API Rate Limiting & Throttling • Secure-by-Default Platform Design • GDPR / EU Regulatory Assessment

Quality Engineering — JUnit 5 & Mockito • Testcontainers Integration Testing • Property-Based Testing (jqwik, fast-check) • Vitest & Testing Library • Playwright • MSW • Load Testing (Locust) • JaCoCo & SonarQube • Architecture Documentation (arc42, C4, Mermaid) • Technical Writing & Reference Documentation

PROFESSIONAL EXPERIENCE

Solutions Architect — TUI

2017 – Present

- Defined architecture strategy for global sourcing and integration platforms supporting multi-regional travel operations, sizing the platform for high-demand seasonal workloads and setting the scalability and resilience targets the delivery teams build against
- Led the decomposition of monolithic applications into AWS-native microservices with event-driven integration — defining service boundaries, ownership model, deployment topology, and automated release paths — reducing deployment time by 60% and improving engineering delivery throughput
- Architected high-availability distributed systems achieving 99.99% uptime and improving peak-load scalability by 40% across business-critical services, addressing failure isolation, redundancy, and capacity headroom as first-class design concerns rather than post-launch fixes
- Established architecture governance practices — design reviews, architecture standards, reusable integration patterns, and engineering best practices — adopted across multiple global teams, turning one-off solutions into documented patterns other teams can apply without re-litigating the decision
- Influenced technical direction across Product, Engineering, Security, and DevOps organizations, translating enterprise strategy into concrete platform decisions and making the operational and security implications of those decisions explicit to non-technical stakeholders
- Established asynchronous messaging standards on Kafka and SNS/SQS — event contracts, consumer isolation, failure handling, and replay — removing synchronous coupling between domains, improving resilience and scalability, and containing the blast radius of downstream outages
- Defined API and integration strategy for platform services, standardizing REST contracts, versioning, and backward-compatibility rules so consuming teams could evolve independently of provider release cycles
- Maintained enterprise architecture documentation and application landscape governance using LeanIX, improving transparency, dependency visibility, and alignment across IT domains
- Evaluated architectural trade-offs, scalability risks, and long-term platform evolution strategies — including cost, operability, and migration effort — to support sustainable business growth and avoid decisions that are cheap to make and expensive to reverse
- Mentored engineers and architects through design reviews, pairing on architectural decisions, and written guidance, contributing to technical leadership development across teams

Environment: AWS • Kafka • SNS/SQS • Java • Spring Boot • REST APIs • Microservices • Distributed Systems • CI/CD • LeanIX

Application Architect — Hotelbeds

2015 – 2017

- Designed and scaled backend services processing millions of daily transactions across high-availability travel distribution platforms, with capacity, latency, and failure behaviour designed for peak distribution load rather than average traffic
- Led modernization of legacy systems into modular microservices-based architectures, defining service decomposition, data ownership, and integration contracts to improve scalability and maintainability
- Improved platform performance by up to 30% through architectural optimization and distributed system enhancements, targeting the structural bottlenecks — call topology, data access paths, and caching strategy — rather than isolated code tuning
- Defined reusable frameworks, integration standards, and development practices, giving multiple teams a common foundation and improving engineering consistency and delivery efficiency

- Collaborated with engineering and business stakeholders to prioritize architectural investment, support scalable platform evolution, and sustain operational reliability under continuous change

Environment: *Java • Spring • REST APIs • Microservices • Distributed Systems • High Availability*

Application Architect — Onwhyon

2014 – 2015

- Architected a digital analytics platform integrating SEO, social media, and analytics systems into a unified reporting ecosystem, defining the integration model that made heterogeneous third-party sources queryable as one dataset
- Designed scalable data processing pipelines combining APIs, crawling technologies, and analytics engines, accounting for rate limits, partial failures, and highly variable source data quality
- Improved reporting scalability and integration reliability for multi-source analytical workloads, reducing the operational cost of onboarding each additional data source

Environment: *APIs • Data Processing • Analytics Platforms • Distributed Processing*

Senior Consultant / Software Architect — Trentiserv

2010 – 2014

- Delivered architecture solutions for enterprise banking systems, including SEPA-related platforms and financial integrations, working within strict correctness, traceability, and regulatory constraints
- Designed high-throughput systems for processing large volumes of financial and transactional data, with emphasis on data integrity, reconciliation, and predictable behaviour under batch and peak load
- Advised stakeholders on scalability, integration strategy, and architectural decision-making for enterprise platforms, making risk and long-term cost explicit in each recommendation
- Collaborated with technical and business teams to deliver resilient and maintainable financial systems that remained supportable well beyond initial delivery

Environment: *Enterprise Integration • Financial Systems • High-Throughput Processing • Distributed Architectures*

CERTIFICATIONS

- AWS Architecting
- AWS Certified Cloud Practitioner
- Scrum Master Certified

EDUCATION

MBA — EUDE Business School

Electronic Engineering — UNEXPO

LANGUAGES

Spanish — Native

English — Professional Working Proficiency